EEST N°1 De Vicente Lopez
Laboratorio de Programación (LPR) — 5° 3° A

Informe de Proyecto

Calculadora de Área Modular — Actividad 3: Enfoque Modular y Diseño Top-Down (C++)

Alumno: Lucas Del Pino

Curso: 5° 3° A

Docente: Prof. York, Prof. Ganduglia

Modalidad: Trabajo individual

Fecha de entrega: 15 de octubre de 2026

Repositorio GitHub: https://github.com/Pin0l33/calculoarea.git

1. Introducción

El presente informe documenta el desarrollo de la Actividad 3 de la materia Laboratorio de Programación (LPR), correspondiente al 1° cuatrimestre de 2026. El objetivo del trabajo consistió en aplicar un enfoque estructurado y de diseño Top-Down (de lo general a lo particular) para construir, en C++, un programa modular que calcula el área de un círculo mediante la fórmula $A = \pi \cdot r^2$, con un filtro de consistencia que valida el radio ingresado y una diana de tiro simulada en ASCII como bonus de gamificación.

2. Estructura del repositorio

El proyecto fue organizado en GitHub siguiendo la estructura de carpetas solicitada por la cátedra:

`/calculoarea/ → .gitignore, LICENSE, README.md, docs/, src/, capturas/`

La carpeta `src/` contiene el código fuente (`main.cpp`), `docs/` este informe en PDF y `capturas/` las evidencias (`compilacion_area.png` y `diagrama_flujo.png`).

3. Captura del código fuente (Visual Studio Code)

A continuación se muestran los bloques completados del código fuente: el prototipo, la llamada a la función con la salida de datos y la definición de `calcularAreaCirculo`. El nombre real del alumno figura en el encabezado y en la salida del programa.

```cpp
#include <iostream>

using namespace std;

// ==========================================================
// 1. PROTOTIPO DE LA FUNCIÓN (Firma de la función)
// Le avisamos al compilador la existencia del módulo de cálculo antes del main
// ==========================================================
double calcularAreaCirculo(double radio);

int main() {
    double radioEstudiante;

    // ==========================================================
    // REGLA OBLIGATORIA (EVITA EL PLAGIO):
    // Nombre y Apellido reales en la salida de pantalla.
    // ==========================================================
    cout << "=====================================================" << endl;
    cout << "  CALCULADORA DE AREA MODULAR - ESTUDIANTE: Lucas Del Pino  " << endl;
    cout << "=====================================================" << endl;

    cout << "=> Ingrese el radio del circulo/diana (en cm): ";
    cin >> radioEstudiante;

    // FILTRO DE CONSISTENCIA (Validación del dato geométrico)
    if (radioEstudiante <= 0) {
        cout << "[ERROR] El radio debe ser un valor positivo y mayor a cero." << endl;
        return 1; // Salida controlada del sistema indicando error en consola
    }

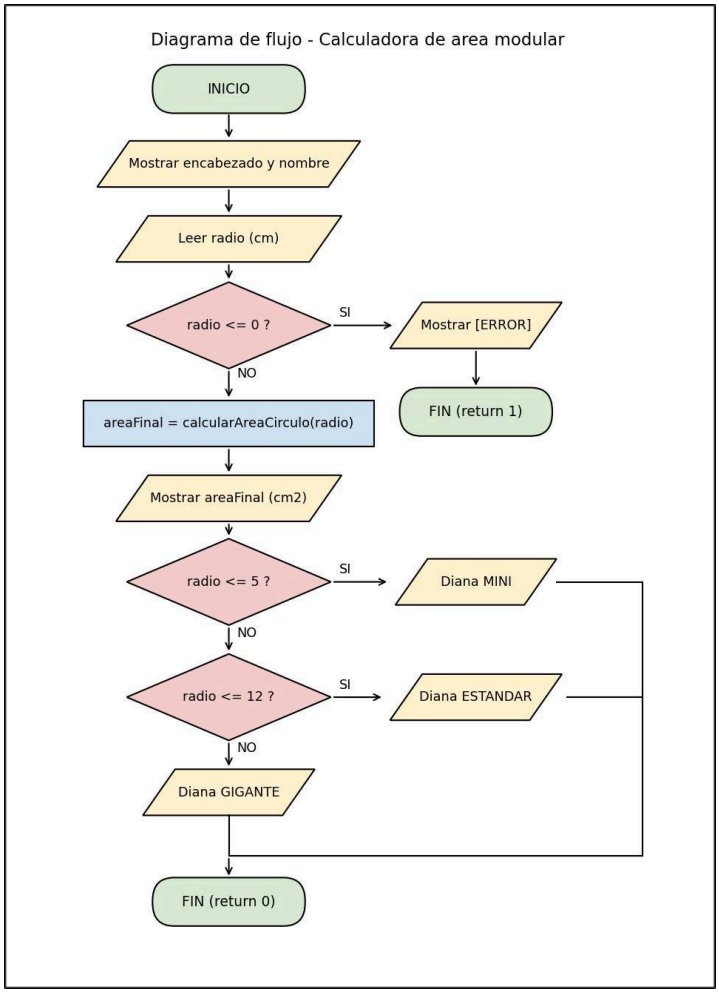
    // ==========================================================
    // 2. LLAMADA / INVOCACIÓN DE LA FUNCIÓN
    // Se invoca al módulo 'calcularAreaCirculo' pasándole 'radioEstudiante'
    // y se guarda el resultado en la variable decimal 'areaFinal'.
    // ==========================================================
    double areaFinal = calcularAreaCirculo(radioEstudiante);

    // ==========================================================
    // 3. SALIDA DE DATOS
    // Se muestra en pantalla el área calculada utilizando 'areaFinal'.
    // ==========================================================
    cout << "[SISTEMA] El area del circulo es: " << areaFinal << " cm2" << endl;

    // ==========================================================
    // BONUS GAMIFICACIÓN: RENDER DE LA DIANA EN CONSOLA
    // ==========================================================
    cout << "\n[SISTEMA] Dibujando escala de la Diana en la RAM..." << endl;
    if (radioEstudiante <= 5.0) {
```

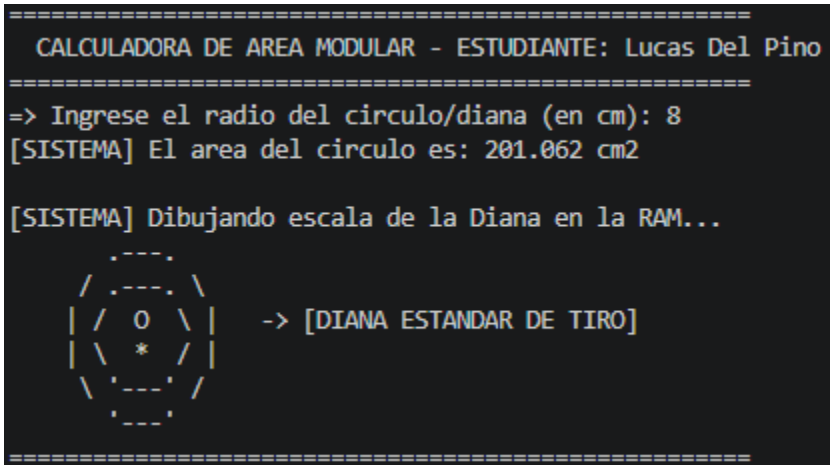
4. Diagrama de flujo

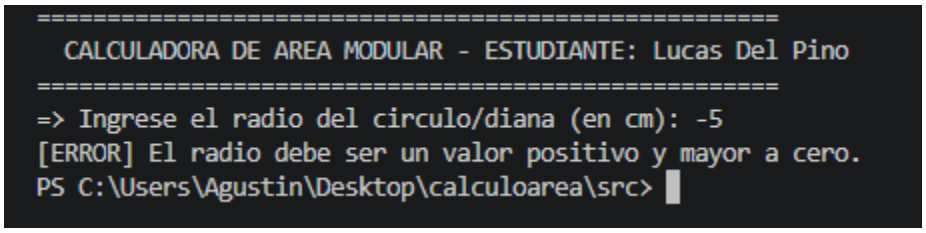
El diagrama muestra el recorrido del programa: lectura del radio, filtro de consistencia, invocación del módulo de cálculo, salida del área y elección de la diana según el radio.



5. Captura de la ejecución (Terminal)

Se compiló con `g++ src/main.cpp -o src/area` y se ejecutó el programa dos veces: con un radio válido (8 cm → área de 201.062 cm², diana estándar) y con un radio inválido (-5), que activa el filtro de consistencia.





6. Cuestionario técnico de control

Pregunta 1: El prototipo de la función

El compilador lee el archivo de arriba hacia abajo. En `main()` aparece la llamada a `calcularAreaCirculo(radioEstudiante)`, pero la definición de la función está después del `main`. Si se borra el prototipo (línea 22 de nuestro archivo), en ese punto el compilador todavía no conoce ese nombre: no sabe qué tipo de dato devuelve ni cuántos parámetros recibe, así que no puede verificar la llamada. En la prueba realizada el compilador detuvo el proceso con el error `'calcularAreaCirculo' was not declared in this scope`. El prototipo resuelve esto sin obligarnos a mover la función arriba del `main`, y así se conserva el orden Top-Down: primero la visión general y después los detalles.

Pregunta 2: El filtro de consistencia

Un círculo de radio 0 cm es un punto: no tiene superficie, y el área da 0, un resultado que no representa una figura real. Un radio negativo no tiene sentido físico, porque las longitudes no pueden ser negativas. Lo más peligroso es que la fórmula no avisa: como $(-5)^2 = 25$, el programa sin filtro calculó un área de 78.5398 cm² (la misma que para un radio de 5 cm) y además dibujó la diana mini, ya que -5 es menor que 5. Es decir, entrega un resultado aparentemente válido pero

falso. Controlar el dato de entrada evita estos errores silenciosos, y es la base de la programación defensiva: ningún cálculo debe confiar en que el usuario ingresó un valor correcto.

Pregunta 3: Proteger constantes en RAM

Declarar un valor con `const` hace que el compilador impida modificarlo. Al probar con `PI = 5.5;` dentro de la función, la compilación se detuvo con el mensaje `assignment of read-only variable 'PI'`. En un sistema crítico, como el software de un avión o un equipo médico, un valor que cambia por accidente (un error de tipeo, como escribir = en lugar de ==, o una modificación involuntaria en otra parte del programa) produciría cálculos incorrectos con consecuencias graves y muy difíciles de rastrear. Con una constante, ese error se detecta en la compilación y no durante el funcionamiento real. Además, `const` documenta la intención del programador: deja claro que ese valor es fijo y facilita el mantenimiento y la revisión del código por otras personas.

7. Conclusión

El desarrollo de esta actividad permitió aplicar el diseño Top-Down y la modularización en C++: separar el cálculo en una función con prototipo, parámetro y retorno; proteger el valor de π con `const`; y validar los datos de entrada antes de operar con ellos. El trabajo también mostró cómo modularizar facilita encontrar errores y reutilizar código, y cómo el compilador ayuda a detectar problemas antes de ejecutar el program